CO4 – AT3

Comparative Analysis

REPORT:

Shortest Path Comparison – Dijkstra vs Bellman-Ford

Aim

To implement Dijkstra's and Bellman-Ford algorithms for finding shortest paths in a weighted graph, compare their correctness and complexity, and analyze why Dijkstra's algorithm fails when negative-weight edges are present.

Problem Statement

Given a weighted graph, the objective is to find the shortest distance from a source vertex to every other vertex.

The graph may contain negative-weight edges. Therefore, both Dijkstra's and Bellman-Ford algorithms are considered and compared.

The main objectives are:

- Calculate shortest paths.
- Compare both algorithms.
- Analyze the effect of negative edges.
- Study time and space complexity.
- Understand the limitations of each algorithm.
- Select the appropriate algorithm based on graph properties.

Input

The program uses:

- Number of vertices = 5

- Number of edges = 8
- Source vertex = 0
- Weighted directed edges.

The graph contains a negative edge:

1 → 4 = -4

2 → 3 = -3

3 → 1 = -2

Output

The program calculates the shortest distance from source vertex 0 to every other vertex.

Example:

Source → 0 = 0

Source → 1 = 2

Source → 2 = 7

Source → 3 = 4

Source → 4 = -2

6. Dijkstra's Algorithm

Dijkstra's algorithm finds the shortest paths from a single source to all other vertices.

It follows a greedy approach.

Steps

1. Set the source distance to 0.
2. Set all other distances to infinity.
3. Select the unvisited vertex with the smallest distance.
4. Mark it as visited.

5. Relax its neighboring edges.
6. Repeat until all reachable vertices are processed.

Main Limitation

Dijkstra assumes that once the closest unvisited vertex is selected, its shortest distance is final.

This assumption is valid only when edge weights are non-negative.

Therefore, Dijkstra is not suitable for graphs containing negative-weight edges in general.

Bellman-Ford Algorithm

Bellman-Ford also calculates shortest paths from a single source but can handle negative-weight edges.

Steps

1. Initialize the source distance to 0.
2. Initialize all other distances to infinity.
3. Relax every edge.
4. Repeat the relaxation V - 1 times.
5. Perform one additional relaxation pass.
6. If a distance can still be reduced, a negative-weight cycle exists.

Relaxation

For an edge:

$u \rightarrow v$

with weight w, relaxation is:

IF dist[u] + w < dist[v]

dist[v] = dist[u] + w

This operation is the main step in both algorithms.

Why Dijkstra Can Fail

Dijkstra selects the currently closest vertex and assumes that its distance cannot later be improved.

Consider:

$A \rightarrow B = 2$

$A \rightarrow C = 5$

$C \rightarrow B = -10$

Starting from A:

Distance(A) = 0

Distance(B) = 2

Distance(C) = 5

Dijkstra may finalize B with distance 2.

However, the path:

$A \rightarrow C \rightarrow B$

has cost:

$5 + (-10) = -5$

Therefore, the actual shortest distance to B is:

-5

but Dijkstra has already finalized B as 2.

This demonstrates why Dijkstra cannot guarantee correct results when negative-weight edges exist.

Bellman-Ford Solution to the Same Problem

Bellman-Ford repeatedly relaxes all edges.

Therefore, even if a negative edge produces a better path later, the improvement can propagate through the graph.

For the example:

$A \rightarrow C \rightarrow B$

Bellman-Ford eventually updates:

$B = -5$

Hence, Bellman-Ford correctly handles negative-weight edges.

Comparison

Feature	**Dijkstra**	**Bellman-Ford**
Approach	Greedy	Dynamic relaxation
Negative edges	Not supported safely	Supported
Negative cycles	Cannot handle	Can detect
Time Complexity	$O(V^2)$ or $O((V+E) \log V)$	$O(VE)$
Space Complexity	$O(V)$ to $O(V+E)$	$O(V)$
Speed	Faster	Slower
Best use	Non-negative weights	Negative or mixed weights
Single-source shortest path	Yes	Yes

Time Complexity

Dijkstra

Using an adjacency matrix:

$O(V^2)$

Using an adjacency list and priority queue:

$O((V + E) \log V)$

For sparse graphs, the priority-queue implementation is generally more efficient.

Bellman-Ford

Every edge is relaxed up to V - 1 times.

Therefore:

$O(VE)$

This makes Bellman-Ford slower than Dijkstra for many graphs.

Space Complexity

Dijkstra

The distance and visited arrays require:

$O(V)$

With an adjacency matrix, the graph itself requires:

$O(V^2)$

Therefore, including the graph representation:

$O(V^2)$

for the matrix implementation used here.

Bellman-Ford

The distance array requires:

$O(V)$

The edge list requires:

O(E)

Therefore, including the graph representation:

O(V + E)

Correctness

Dijkstra

Dijkstra is correct when all edge weights are non-negative.

It is not guaranteed to produce the correct shortest paths when negative-weight edges are present.

Bellman-Ford

Bellman-Ford is correct for graphs containing negative-weight edges, provided there is no reachable negative-weight cycle.

It also has the additional ability to detect negative-weight cycles.

Negative Weight Cycle

A negative-weight cycle is a cycle whose total edge weight is negative.

Example:

A → B = 2

B → C = -4

C → A = 1

Total:

2 + (-4) + 1 = -1

Because the total is negative, repeatedly travelling around this cycle can continuously reduce the path cost.

In such a graph, a finite shortest path may not exist.

Bellman-Ford detects this by performing an additional relaxation after V - 1 iterations.

Performance Analysis

Dijkstra generally performs better because it does not repeatedly examine every edge.

Bellman-Ford performs more relaxation operations, giving it a higher worst-case time complexity.

For example:

Dijkstra → $O(V^2)$

Bellman-Ford → $O(VE)$

Therefore:

- Small graph with non-negative weights → Dijkstra is suitable.
- Large sparse graph with non-negative weights → Dijkstra with a priority queue is suitable.
- Graph with negative weights → Bellman-Ford should be used.
- Graph requiring negative-cycle detection → Bellman-Ford is preferred.

Algorithm Selection

The choice depends mainly on edge weights.

Use Dijkstra when:

- All edge weights are non-negative.
- Fast execution is important.
- The graph is large.
- Negative edges are not present.

Use Bellman-Ford when:

- Negative edges are present.
- Negative-cycle detection is required.

- Correctness is more important than speed.

Result

Both Dijkstra's and Bellman-Ford algorithms were implemented and compared.

The analysis shows that:

- Dijkstra is faster but requires non-negative edge weights.
- Bellman-Ford is slower but supports negative-weight edges.
- Bellman-Ford can detect negative-weight cycles.
- Dijkstra may produce incorrect results when a negative edge creates a better path after a vertex has already been finalized.